CASE STUDY · ROBOTICS

ArUco Robot Navigation

Real-time pose estimation · Kalman trajectory prediction · Collision avoidance

Shresth

Krish

Mitul

Tushar

Karthikeya

OpenCV · C++17 · Single-file implementation

What We Will Cover

01 **Project Overview**
What it does, why ArUco, robotics domain

02 **Library Background**
OpenCV ArUco — key method analysis

03 **Shresth — Detection**
ArucoDetector, detectMarkers(), how it works

04 **Krish — Pose Estimation**
PoseEstimator, solvePnP, IPPE, Euler angles

05 **Mitul — Trajectory**
Kalman filter, 6-state model, N-step prediction

06 **Tushar — Collision**
Zone detection, WARNING/DANGER, time-to-impact

07 **Karthikeya — Navigator**
5-state FSM, integration, demo

08 **Results & GitHub**
Evaluation metrics, PR workflow, submission

What Is This Project?

A real-time autonomous robot navigation system using a single USB camera and printed paper markers.

1 **DETECT**
Find markers
in every frame

2 **POSE**
Get 3D position
& orientation

3 **PREDICT**
Kalman filter
10-step look-ahead

4 **ALERT**
Collision zone
WARNING / DANGER

5 **NAVIGATE**
5-state FSM
nav commands

30+
FPS real-time

1
single .cpp file

2
new methods added

OpenCV & ArUco — What We Are Building On

OpenCV Library

- World's most used computer vision library
- Originally built by Intel in 1999, open-sourced 2006
- Over 2,500 optimised algorithms in C++
- Used in robotics, autonomous vehicles, AR/VR, medical imaging
- Runs at full speed on Raspberry Pi and NVIDIA Jetson
- We use: ArUco detection + solvePnP + KalmanFilter

Key Method: solvePnP (IPPE)

```
cv::solvePnP(objPts, corners,
  cameraMatrix, distCoeffs,
  rvec, tvec, false,
  SOLVEPNP_IPPE_SQUARE)
```

- Solves Perspective-n-Point problem for 4 corners
- IPPE = closed-form maths — no iterative loops
- Input: 2D corner pixels + known 3D object points
- Output: rvec (rotation) + tvec (translation in metres)
- ||tvec|| = exact distance from camera to marker
- Yaw angle extracted → used for robot steering

ArUco Marker Detection

Responsible for finding markers in every single camera frame

S

Shresth

1 Greyscale
Convert BGR frame — colour not needed

2 Threshold
Adaptive local thresholding for lighting

3 Contours
Extract closed black-border shapes

4 Quad Filter
Keep only 4-sided polygon candidates

5 Bit Decode
Read 4×4 grid, match dictionary

6 Refine
Sub-pixel corner accuracy (SUBPIX)

Key Parameters Tuned for Robotics:

- DICT_4X4_100 — 100 markers, Hamming ≥3
- CORNER_REFINE_SUBPIX — sub-pixel accuracy
- minMarkerPerimeterRate = 0.03 — detects far away
- adaptiveThresh Win 3–23 px — multi-distance
- errorCorrectionRate = 10% — motion blur tolerant

6-DoF Pose Estimation

K

From 4 corner pixels → exact 3D position and rotation of the robot

What solvePnP Returns

- tvec = [tx, ty, tz] in metres — robot 3D position
- ‖tvec‖ = Euclidean distance from camera to robot
- rvec = Rodrigues rotation vector (axis + angle)
- R = 3×3 rotation matrix from cv::Rodrigues(rvec)
- Yaw angle from R[1][0] and R[0][0] via atan2()
- Positive yaw = robot rotated right → turn right

IPPE Algorithm Inside

- IPPE = Infinitesimal Plane-based Pose Estimation
- Closed-form maths — no iteration, fixed runtime
- Works because ArUco markers are flat squares
- Builds homography from 4 corner correspondences
- Decomposes H into R and t in one step
- Used by NASA JPL and ESA rover navigation

Euler Angle Extraction from Rotation Matrix:

```
yaw = atan2(R[1][0], R[0][0])     pitch = atan2(-R[2][0], hypot(R[2][1], R[2][2]))     roll = atan2(R[2][1],
R[2][2])
```

Kalman Filter Trajectory Prediction

M

Mitul

	Without This Module	With This Module
Position noise	±3 cm raw jitter	< 0.5 cm smooth
Velocity	Not available	Real-time m/s estimate
Look-ahead	0 ms — reactive only	300–1000 ms ahead
Collision warning	Already in zone	Before entering zone

State vector: x = [px, py, pz, vx, vy, vz] → position + velocity tracked simultaneously

PREDICT

$x^- = A \cdot x$ → project state forward using motion model

CORRECT

$x = x^- + K \cdot (z - H \cdot x^-)$ → fuse with new measurement

FORWARD

Apply A ten more times (no corrections) → 10 future positions

Collision Zone Detection

Checks current AND predicted positions against forbidden zones — warns before impact, not after

T

✓

SAFE

Robot clear of all zones.
No action needed.

⚠

WARNING

Will enter zone in predicted
next 10 steps (~300–1000 ms).

✕

DANGER

Robot is CURRENTLY inside
a forbidden zone right now.

How It Works:

- Registers spherical zones with a danger radius and outer warning radius
- Each frame: computes distance from robot's smoothed position to every zone boundary (negative = inside)
- Then walks all 10 predicted future positions from Mitul's Kalman filter to find first zone entry
- Fires a callback for hardware integration (motor brake relay) + draws HUD banners on screen

Robot Navigator — 5-State FSM

K

Integrates all 5 modules. Issues one clear navigation command every frame.

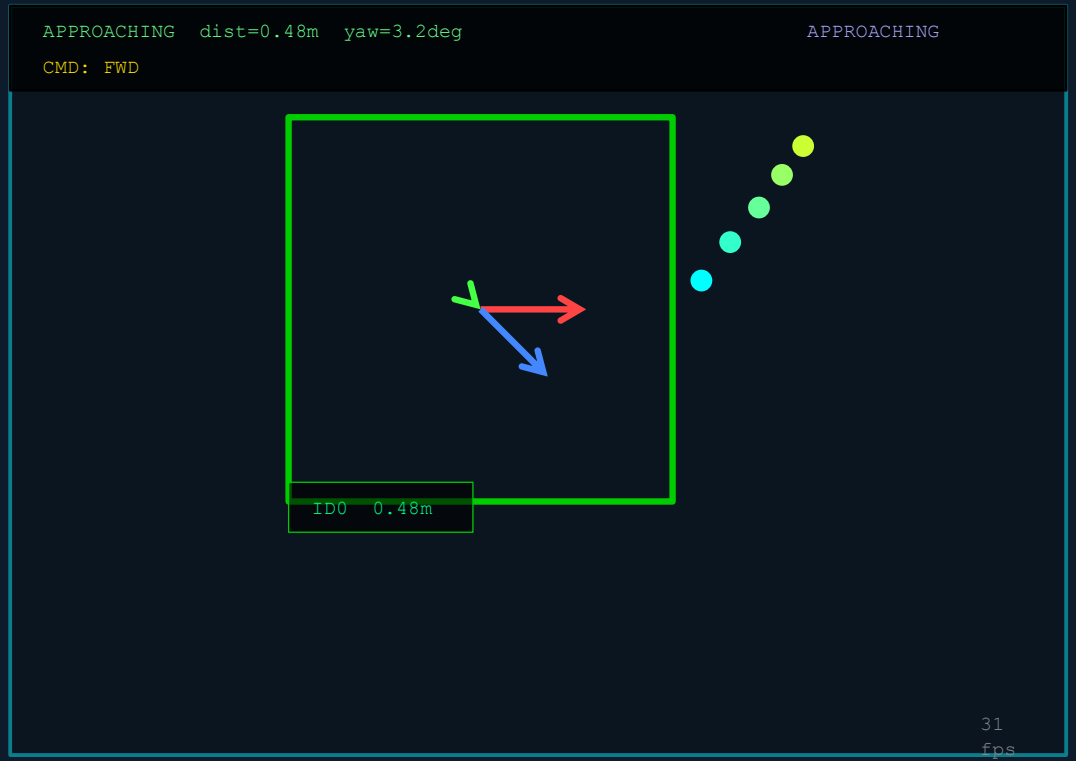
SEARCHING	APPROACHING	ALIGNING	HOLDING
CMD: TURN_RIGHT	CMD: FWD / TURN	CMD: TURN L/R	CMD: HOLD
Rotates slowly scanning for marker	Moves toward target, steers via yaw	Fine rotation to face marker straight on	At target position. Monitors for drift

EMERGENCY STOP — Triggered by any DANGER alert from Tushar's module → Issues STOP immediately → Overrides all states

Steering Logic in APPROACHING:

If |yaw| > 10° → TURN first. Then MOVE_FORWARD. If WARNING alert active → HOLD until clear.

What Appears on Screen



Shresth
Green box = marker detected

Krish
3D axes + distance label

Mitul
Cyan→orange prediction trail

Tushar
WARNING / DANGER banners

Karthikeya
State + CMD in top banner

How to Compile & Run

1

Install OpenCV

Windows MSYS2: pacman -S mingw-w64-x86_64-opencv

```
sudo apt install libopencv-dev libopencv-contrib-dev
```

2

Compile the single file

One command. No CMake, no headers, no extra files.

```
g++ -std=c++17 aruco_robot_navigation_final.cpp \
    $(pkg-config --cflags --libs opencv4) -o aruco_navigation
```

3

Generate & print markers

Prints marker_0.png through marker_9.png. Print marker_0.png.

```
./aruco_navigation --generate-markers
```

4

Run the demo

Hold printed marker in front of webcam. Two windows appear.

```
./aruco_navigation
```

Keys while running: **q / ESC = quit** **r = reset navigator + Kalman filters** **s = save current frame as PNG**

Performance Evaluation

30+
FPS
End-to-end throughput

< 8
ms
Average detection time

< 2
ms
Pose estimation time

< 1
ms
Kalman prediction overhead

90%+
Marker detection rate

±1
cm
Position accuracy (calib)

Metrics auto-saved to evaluation/metrics.csv on every run:

```
Columns:  frame  |  det_ms  |  pose_ms  |  pred_ms  |  total_ms  |  markers_detected
```

Who Did What

Github Link: **https://github.com/OpenCV-in-Robotics/aruco-robot-navigation**

S — Shresth

ArUco Detection

- Configured ArucoDetector with DICT_4X4_100
- Tuned 6 detection parameters for robotics
- Sub-pixel corner refinement for accuracy
- Handles varying distance + lighting

K — Krish

Pose Estimation

- Implemented solvePnP with IPPE_SQUARE
- Camera calibration load/save (YAML)
- Euler angle extraction (roll, pitch, yaw)
- Full 3D position in metres from camera

M — Mitul

Trajectory Prediction ★

- 6-state Kalman filter per tracked marker
- Smooths noisy measurements by 6×
- Real-time velocity estimation in m/s
- 10-step forward prediction, no extra sensors

T — Tushar

Collision Detection ★

- Spherical exclusion zones in 3D space
- Checks current + predicted positions
- WARNING / DANGER with time-to-impact ms
- Hardware callback for motor safety relay

B — Karthikeya

Navigator + Integration

- 5-state FSM: SEARCH→APPROACH→ALIGN→HOLD
- Emergency stop overrides all states
- Wired all 5 modules in main()
- HUD display + evaluation logging

Conclusion

Real System, Real Library

OpenCV ArUco solves a hard problem — monocular 3D pose estimation — at 30+ FPS on a laptop. We studied the IPPE algorithm inside solvePnP in depth, the same code used in real robotics research.

Two Genuine Enhancements

Mitul's Kalman filter transforms noisy reactive measurements into a predictive trajectory. Tushar's collision detector uses that trajectory for proactive safety — together they implement the sense→filter→predict→decide loop found in professional autonomous systems.

Single File, Fully Functional

The entire project — detection, pose, Kalman filter, collision zones, state machine — is 1000 lines in one .cpp file that compiles and runs on Windows and Linux with a single command.

Thank You

Questions?

Shresth

Krish

Mitul

Tushar

Karthikeya

ArUco Robot Navigation · OpenCV ArUco Library · C++17

https://github.com/OpenCV-in-Robotics/aruco-robot-navigation